

# Code Review Report

Language: java | Review mode: hybrid | Model: llama-3.3-70b-versatile | Generated: 2026-08-13 17:27 UTC

## Overall Assessment

The code review has identified 9 findings with a risk score of 32, indicating a significant security risk. The majority of the findings are critical, highlighting the need for immediate attention to prevent potential security breaches.

Merge readiness: Not ready for merge due to critical and high-severity findings that require immediate attention to prevent potential security breaches.

Code health: 0/100

## Findings Summary

Total findings: 9

Risk score: 32

Severity breakdown: Low 0, Medium 0, High 4, Critical 5

| Type                        | Severity | Line | Message                                                                 | Recommendation                                             |
|-----------------------------|----------|------|-------------------------------------------------------------------------|------------------------------------------------------------|
| Hardcoded Secret            | High     | 3    | String password = "admin123";                                           | Load secrets from environment variables or a secure vault. |
| Hardcoded Secret            | High     | 4    | String api_key = "ABC-123-SECRET";                                      | Load secrets from environment variables or a secure vault. |
| SQL Injection               | Critical | 5    | String query = "SELECT * FROM " + table + " WHERE name = " + name + ""; | Use prepared statements and parameter binding.             |
| Command Injection           | Critical | 7    | Runtime.getRuntime().exec("cmd /c dir " + name);                        | Avoid shell execution with user-controlled input.          |
| Hardcoded Password          | High     | 3    | String password = "admin123";                                           | Store passwords in environment variables.                  |
| Hardcoded API Key           | High     | 4    | String api_key = "ABC-123-SECRET";                                      | Store secrets securely.                                    |
| Dangerous Dynamic Execution | Critical | 7    | Runtime.getRuntime().exec("cmd /c dir " + name);                        | Avoid dynamic code execution.                              |
| SQL Injection               | Critical | 10   | Unsafe SQL string formatting.                                           | Use prepared statements.                                   |
| Command Injection           | Critical | 18   | Runtime.exec with unsanitized input.                                    | Avoid shell execution with user input.                     |

## Remediation Roadmap

| Finding       | Severity | Impact                   | Suggested Action                                                                 |
|---------------|----------|--------------------------|----------------------------------------------------------------------------------|
| SQL Injection | Critical | High risk of data breach | Use prepared statements with parameter binding to prevent SQL injection attacks. |

| Finding                     | Severity | Impact                                 | Suggested Action                                                                        |
|-----------------------------|----------|----------------------------------------|-----------------------------------------------------------------------------------------|
| Command Injection           | Critical | High risk of system compromise         | Avoid shell execution with user-controlled input and use parameterized queries instead. |
| Dangerous Dynamic Execution | Critical | High risk of command injection attacks | Replace Runtime.getRuntime().exec() with a static method call or a parameterized query. |
| Hardcoded Secret            | High     | High risk of credential exposure       | Load secrets from environment variables or a secure vault instead of hardcoding them.   |
| Hardcoded Password          | High     | High risk of credential exposure       | Store passwords in environment variables instead of hardcoding them.                    |
| Hardcoded API Key           | High     | High risk of API key exposure          | Store API keys securely using environment variables or a secrets manager.               |

## Source Code Snapshot

```

public class VulnerableSecurity {
    public void runQuery(String name, String table) {
        String password = "admin123";
        String api_key = "ABC-123-SECRET";
        String query = "SELECT * FROM " + table + " WHERE name = '" + name + "'";
        System.out.println(query + password + api_key);
        Runtime.getRuntime().exec("cmd /c dir " + name);
    }
}

```